Assignment (Module III): Core Java

Task 1: ATM Withdrawal System – Handling Insufficient Balance
Scenario:
In a banking system, when a user tries to withdraw more money than their account balance, the system should throw an exception and display an appropriate message.
Program Structure:
• Class: ATM
o Field: balance
o Method: withdraw(double amount) — throws exception if amount > balance
• Exception: Use throw and try-catch
• Use finally block to display "Transaction Completed".

Solution :

```
import java.util.Scanner;

public class ATM {
    double balance;

    ATM(double balance) {
        this.balance = balance;
    }

    void withdraw(double amount) throws Exception {
        if (amount > balance) {
            throw new Exception("Insufficient Balance");
        }
        balance -= amount;
        System.out.println("Withdrawal Successful");
        System.out.println("Remaining Balance: " + balance);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

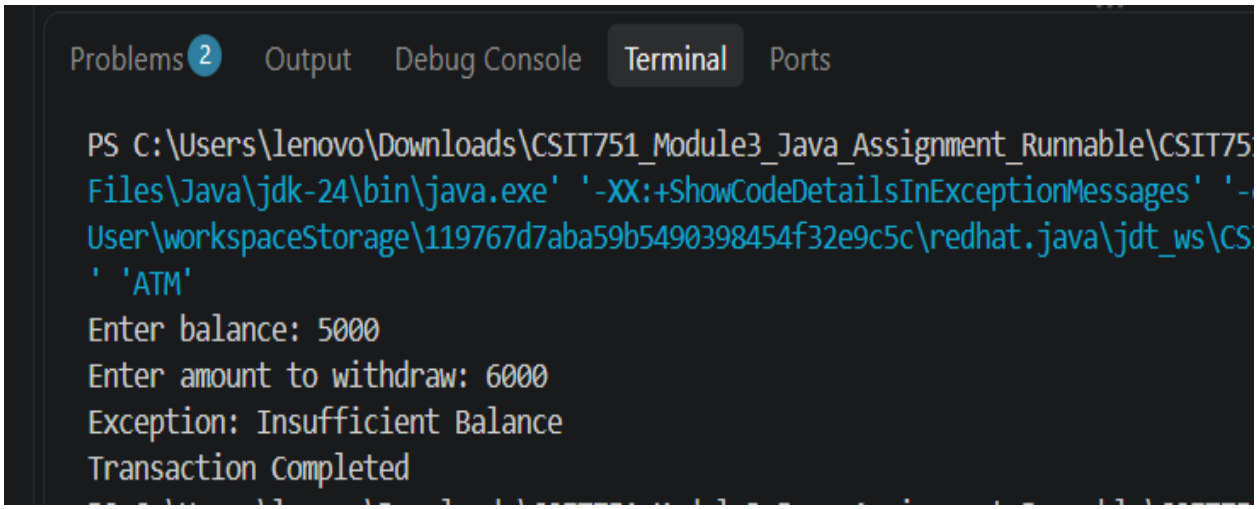
        System.out.print("Enter balance: ");
        double balance = sc.nextDouble();
```

```
        System.out.print("Enter amount to withdraw: ");
        double amount = sc.nextDouble();

        ATM atm = new ATM(balance);

        try {
            atm.withdraw(amount);
        } catch (Exception e) {
            System.out.println("Exception: " + e.getMessage());
        } finally {
            System.out.println("Transaction Completed");
        }

        sc.close();
    }
}
```



Task 2: Student Age Validation – User Defined Exception
Scenario:
While registering students, the age must be above 17. If not, throw a user-defined exception.
Program Structure:
• Class: Student
• User Defined Exception: InvalidAgeException
• Method: registerStudent(int age) — throws InvalidAgeException if age < 17
• Handle using try-catch.

Solution :

```
import java.util.Scanner;

class InvalidAgeException extends Exception {
    InvalidAgeException(String message) {
        super(message);
    }
}

public class Student {
    void registerStudent(int age) throws InvalidAgeException {
        if (age <= 17) {
            throw new InvalidAgeException("Age must be above 17 for
registration");
        }
        System.out.println("Student registered successfully");
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

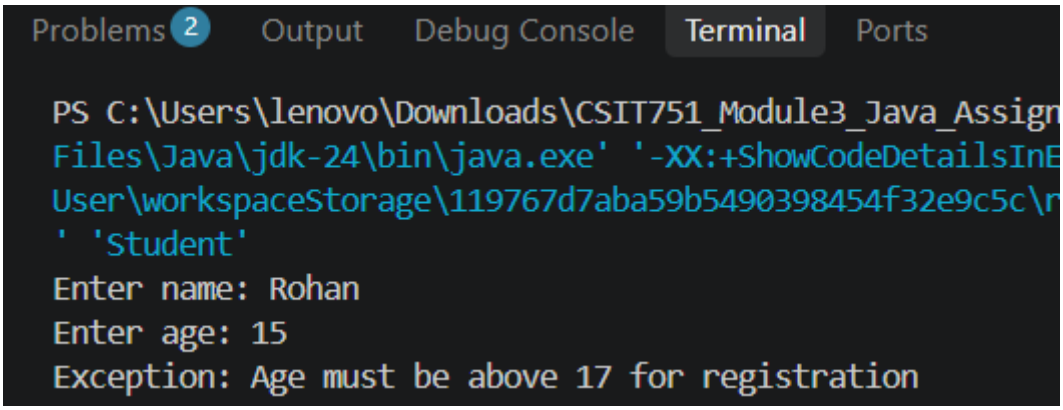
        System.out.print("Enter name: ");
        String name = sc.nextLine();

        System.out.print("Enter age: ");
        int age = sc.nextInt();

        Student student = new Student();

        try {
            student.registerStudent(age);
        } catch (InvalidAgeException e) {
            System.out.println("Exception: " + e.getMessage());
        }
```

```
            sc.close();
        }
}
```


Problems 2
Output
Debug Console
Terminal
Ports
PS C:\Users\lenovo\Downloads\CSIT751_Module3_Java_Assign
Files\Java\jdk-24\bin\java.exe' '-XX:+ShowCodeDetailsInE
User\workspaceStorage\119767d7aba59b5490398454f32e9c5c\r
' 'Student'
Enter name: Rohan
Enter age: 15
Exception: Age must be above 17 for registration

Task 3: Array Division – Handling Arithmetic and Array Index Exception (Multiple Catch)
Scenario:
A program takes two numbers and an array index. It divides array[index] by the number. You need to handle both ArithmeticException (if divisor = 0) and ArrayIndexOutOfBoundsException (if index invalid).
Program Structure:
• Class: ArrayDivision
• Use multiple catch blocks.

Solution :

```
import java.util.Scanner;

public class ArrayDivision {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[] array = {10, 20, 30, 40};

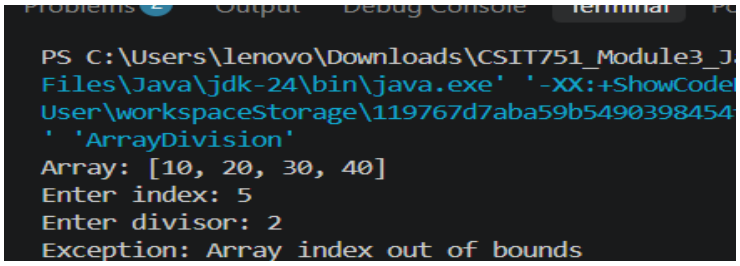
        System.out.println("Array: [10, 20, 30, 40]");

        System.out.print("Enter index: ");
        int index = sc.nextInt();

        System.out.print("Enter divisor: ");
        int divisor = sc.nextInt();

        try {
            int result = array[index] / divisor;
            System.out.println("Result: " + result);
        } catch (ArithmeticException e) {
            System.out.println("Exception: Division by zero not allowed");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Exception: Array index out of bounds");
        }

        sc.close();
    }
}
```



Task 4: Student Marks Entry – Input Validation
Scenario:
If a user enters a negative mark or mark greater than 100, throw an exception. Also, ensure that program always displays a thank you message using finally.
Program Structure:
• Class: StudentMarks
• Method: enterMarks(int marks) — throws exception for invalid range
• Use try-catch-finally.

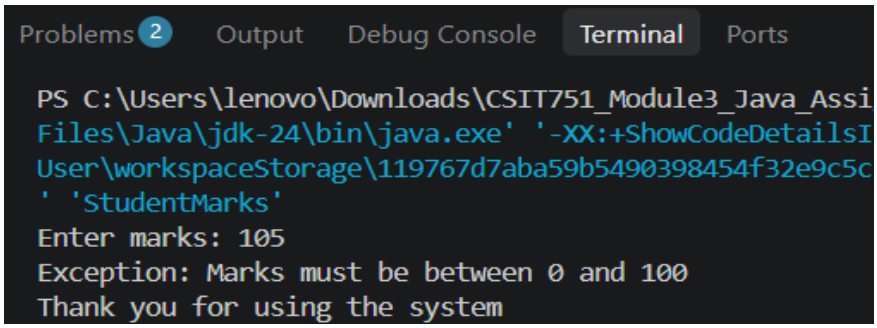
Solution :

```
import java.util.Scanner;

public class StudentMarks {
    void enterMarks(int marks) throws Exception {
        if (marks < 0 || marks > 100) {
            throw new Exception("Marks must be between 0 and 100");
        }
        System.out.println("Marks entered successfully");
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        StudentMarks student = new StudentMarks();
        System.out.print("Enter marks: ");
        int marks = sc.nextInt();
        try {
            student.enterMarks(marks);
        } catch (Exception e) {
            System.out.println("Exception: " + e.getMessage());
        } finally {
            System.out.println("Thank you for using the system");
        }

        sc.close();
    }
}
```



Task 5: University Login System – Null Pointer Handling
Scenario:
In a university login system, if a user tries to log in with a null username, a NullPointerException occurs. Handle this gracefully.
Program Structure:
• Class: UniversityLogin
• Method: login(String username)
• Throw or let NullPointerException occur and handle it using try-catch.

Solution :

```
import java.util.Scanner;

public class UniversityLogin {
    void login(String username) {
        if (username == null) {
            throw new NullPointerException("Username cannot be null");
        }
        System.out.println("Login successful: " + username);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter username: ");
        String input = sc.nextLine();

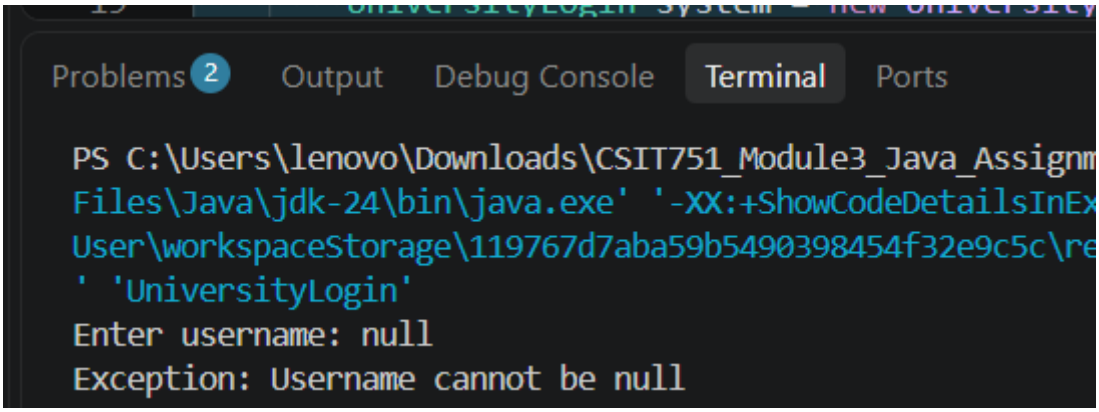
        String username = input.equalsIgnoreCase("null") ? null : input;

        UniversityLogin system = new UniversityLogin();

        try {
            system.login(username);
        } catch (NullPointerException e) {
            System.out.println("Exception: " + e.getMessage());
        }

        sc.close();
```

```
    }
}
```


Problems 2
Output
Debug Console
Terminal
Ports
PS C:\Users\lenovo\Downloads\CSIT751_Module3_Java_Assignm
Files\Java\jdk-24\bin\java.exe' '-XX:+ShowCodeDetailsInEx
User\workspaceStorage\119767d7aba59b5490398454f32e9c5c\re
' 'UniversityLogin'
Enter username: null
Exception: Username cannot be null

Task 6: Online Shopping – Minimum Purchase Amount Exception
Scenario:
An online shopping system requires a minimum cart value of ₹500 for placing an order. If below that, throw a user-defined exception.
Program Structure:
• Class: OnlineShopping
• User Defined Exception: MinimumAmountException
• Method: placeOrder(int amount) — throws exception if amount < 500

Solution :

```
import java.util.Scanner;

class MinimumAmountException extends Exception {
    MinimumAmountException(String message) {
        super(message);
    }
}

public class OnlineShopping {
    void placeOrder(int amount) throws MinimumAmountException {
        if (amount < 500) {
            throw new MinimumAmountException("Minimum cart value must be
₹500");
        }
        System.out.println("Order placed successfully");
    }

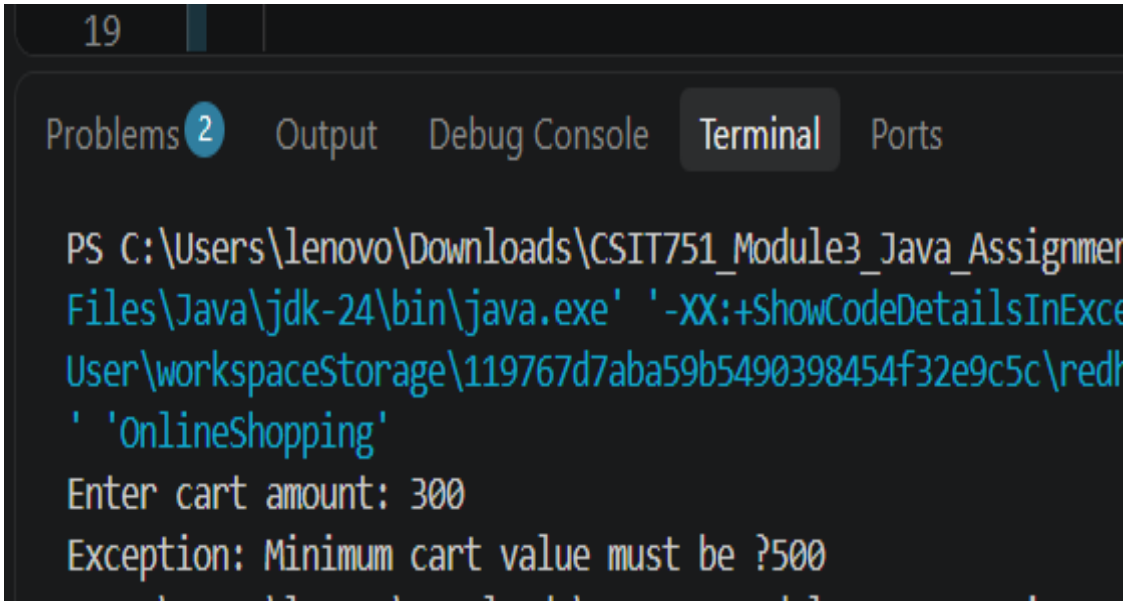
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter cart amount: ");
        int amount = sc.nextInt();

        OnlineShopping shopping = new OnlineShopping();

        try {
            shopping.placeOrder(amount);
        } catch (MinimumAmountException e) {
            System.out.println("Exception: " + e.getMessage());
        }

        sc.close();
    }
}
```


19
Problems 2
Output
Debug Console
Terminal
Ports
PS C:\Users\lenovo\Downloads\CSIT751_Module3_Java_Assignmer
Files\Java\jdk-24\bin\java.exe' '-XX:+ShowCodeDetailsInExce
User\workspaceStorage\119767d7aba59b5490398454f32e9c5c\redh
' 'OnlineShopping'
Enter cart amount: 300
Exception: Minimum cart value must be ?500

Task 7: Railway Ticket Booking – Synchronization of Threads
Scenario:
In a railway booking system, multiple users are trying to book tickets simultaneously. Only a limited number of seats are available. Synchronization must ensure that no two users book the same seat.
Program Structure:
• Class: TicketBooking (Shared Resource)
o Field: availableSeats
o Method: bookSeat(int seats) — synchronized
• Thread classes: User1, User2 extending Thread
• Use synchronization to prevent race conditions.

Solution :

```
import java.util.Scanner;

class TicketBooking {
    private int availableSeats;

    TicketBooking(int availableSeats) {
        this.availableSeats = availableSeats;
    }

    synchronized void bookSeat(int seats) {
        String user = Thread.currentThread().getName();

        if (seats <= availableSeats) {
            System.out.println(user + " booked " + seats + " seat(s)
successfully");
            availableSeats -= seats;
        } else {
            System.out.println(user + " booking failed. Not enough
seats");
        }
    }
}

class User1 extends Thread {
    private final TicketBooking booking;
    private final int seats;

    User1(TicketBooking booking, int seats) {
        this.booking = booking;
        this.seats = seats;
        setName("User1");
    }
```

```
    public void run() {
        booking.bookSeat(seats);
    }
}

class User2 extends Thread {
    private final TicketBooking booking;
    private final int seats;

    User2(TicketBooking booking, int seats) {
        this.booking = booking;
        this.seats = seats;
        setName("User2");
    }

    public void run() {
        booking.bookSeat(seats);
    }
}

public class RailwayBooking {
    public static void main(String[] args) throws InterruptedException {
        Scanner sc = new Scanner(System.in);

        System.out.print("Available Seats: ");
        int availableSeats = sc.nextInt();

        System.out.print("User1 wants to book: ");
        int user1Seats = sc.nextInt();

        System.out.print("User2 wants to book: ");
        int user2Seats = sc.nextInt();

        TicketBooking booking = new TicketBooking(availableSeats);
        User1 u1 = new User1(booking, user1Seats);
        User2 u2 = new User2(booking, user2Seats);

        u1.start();
        u1.join(); // keeps sample-output order predictable
        u2.start();
        u2.join();

        sc.close();
    }
}
```

```
PS C:\Users\lenovo\Downloads\CSIT751_Module3_
Files\Java\jdk-24\bin\java.exe' '-XX:+ShowCod
User\workspaceStorage\119767d7aba59b549039845
' 'RailwayBooking'
Available Seats: 2
User1 wants to book: 1
User2 wants to book: 2
User1 booked 1 seat(s) successfully
User2 booking failed. Not enough seats
```

Task 8: University Printing System – Multiple Threads
Scenario:
A university has multiple printers. Students can send print jobs simultaneously. Each print job runs on a separate thread.
Program Structure:
• Class: PrinterJob implements Runnable
• Create multiple threads for different students.
• Simulate printing using Thread.sleep().

Solution :

```
class PrinterJob implements Runnable {
    private final int jobNumber;
    private final String studentName;

    PrinterJob(int jobNumber, String studentName) {
        this.jobNumber = jobNumber;
        this.studentName = studentName;
    }

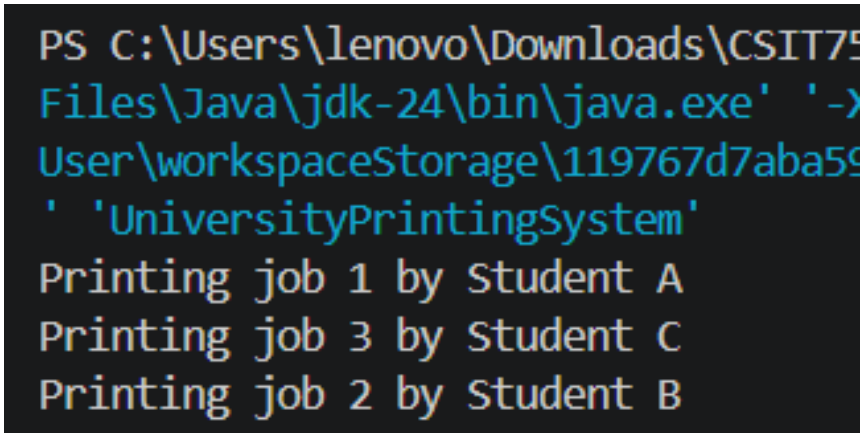
    public void run() {
        System.out.println("Printing job " + jobNumber + " by " +
studentName);

        try {
            Thread.sleep(1000);
        } catch (InterruptedException e) {
            System.out.println("Printing interrupted");
            Thread.currentThread().interrupt();
        }
    }
}

public class UniversityPrintingSystem {
    public static void main(String[] args) throws InterruptedException {
        Thread t1 = new Thread(new PrinterJob(1, "Student A"));
        Thread t2 = new Thread(new PrinterJob(2, "Student B"));
        Thread t3 = new Thread(new PrinterJob(3, "Student C"));
```

```
        t1.start();
        t2.start();
        t3.start();

        t1.join();
        t2.join();
        t3.join();
    }
}
```



Task 9: IoT Sensor Monitoring System
Scenario: An IoT sensor generates a random number every second.
• If even → Square is calculated by one thread.
• If odd → Cube is calculated by another thread.
Menu:
1. Start Simulation
2. Exit
Program Structure:
• RandomNumberGenerator (Thread) → generates numbers.
• SquareCalculator (Thread) → processes even numbers.
• CubeCalculator (Thread) → processes odd numbers.
• Uses Thread.sleep(), multithreading, and conditional checks.

Solution :

```
import java.util.Random;
import java.util.Scanner;

class SquareCalculator extends Thread {
    private final int number;

    SquareCalculator(int number) {
        this.number = number;
    }

    public void run() {
        System.out.println("Square: " + (number * number));
    }
}

class CubeCalculator extends Thread {
    private final int number;

    CubeCalculator(int number) {
        this.number = number;
    }

    public void run() {
        System.out.println("Cube: " + (number * number * number));
    }
}

class RandomNumberGenerator extends Thread {
    public void run() {
        Random random = new Random();
```

```
        try {
            for (int i = 1; i <= 3; i++) {
                int number = random.nextInt(10) + 1;
                System.out.println("Generated: " + number);

                Thread calculator;
                if (number % 2 == 0) {
                    calculator = new SquareCalculator(number);
                } else {
                    calculator = new CubeCalculator(number);
                }

                calculator.start();
                calculator.join();

                Thread.sleep(1000);
            }
        } catch (InterruptedException e) {
            System.out.println("Thread interrupted");
            Thread.currentThread().interrupt();
        }
    }
}

public class IoTSensorMonitoring {
    public static void main(String[] args) throws InterruptedException {
        Scanner sc = new Scanner(System.in);

        while (true) {
            System.out.println("\n1. Start Simulation");
            System.out.println("2. Exit");
            System.out.print("Enter choice: ");

            int choice = sc.nextInt();

            if (choice == 1) {
                RandomNumberGenerator generator = new
RandomNumberGenerator();
                generator.start();
                generator.join();
            } else if (choice == 2) {
                System.out.println("Program exited");
                break;
            } else {
                System.out.println("Invalid choice");
            }
        }
```

```
            sc.close();
        }
}
```

```
PS C:\Users\lenovo\Dow
rs\lenovo\Downloads\CS
a\jdk-24\bin\java.exe'
spaceStorage\119767d7a
sorMonitoring'

1. Start Simulation
2. Exit
Enter choice: 1
Generated: 1
Cube: 1
Generated: 4
Square: 16
Generated: 6
Square: 36
```

Task 10: Bank Transaction System – Thread Priorities
Scenario:
In a banking application, high-value transactions must be processed with higher priority than low-value transactions.
Program Structure:
• Class: BankTransaction extends Thread
• Create threads for different transactions and set priorities: MAX_PRIORITY for high-value, MIN_PRIORITY for low-value.

Solution :

```
import java.util.Scanner;

class BankTransaction extends Thread {
    private final String type;

    BankTransaction(String type) {
        this.type = type;
    }

    public void run() {
        if (type.equals("High-value"))
            System.out.println("High-value transaction processed first");
        else
            System.out.println("Low-value transaction processed later");
    }
}

public class BankTransactionSystem {
    public static void main(String[] args) throws InterruptedException {
        Scanner sc = new Scanner(System.in);

        System.out.print("Transaction1: ₹");
        int t1 = sc.nextInt();

        System.out.print("Transaction2: ₹");
        int t2 = sc.nextInt();

        BankTransaction high = new BankTransaction("High-value");
        BankTransaction low = new BankTransaction("Low-value");

        high.setPriority(Thread.MAX_PRIORITY);
        low.setPriority(Thread.MIN_PRIORITY);

        high.start();
        high.join();
```

```
        low.start();
        low.join();

        sc.close();
    }
}
```

```
PS C:\Users\lenovo\Downloads\CSIT751_Module
rs\lenovo\Downloads\CSIT751_Module3_Java_As
a\jdk-24\bin\java.exe' '-XX:+ShowCodeDetail
spaceStorage\119767d7aba59b5490398454f32e9c
ansactionSystem'
Transaction1: ?5000
Transaction2: ?50000
High-value transaction processed first
Low-value transaction processed later
```